UNIDADE 3

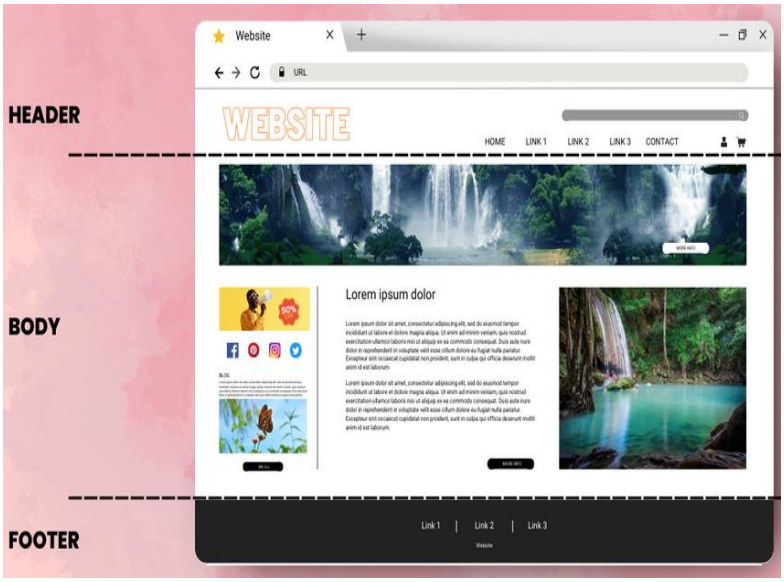
Navegación e Comunicación entre Compoñentes

Obxectivos da unidade

- Integración de **elementos fixos** como ***navbar*** **e** ***footer*** en múltiples vistas
- Comprender e aplicar o ***enrutamento*** en Vue para crear múltiples vistas dentro dunha SPA, rutas básicas, navegación entre compoñentes y parámetros na URL
- Aprender a comunicar compoñentes entre si mediante ***props*** e eventos personalizados, envío de eventos mediante **$*emit*** e introdución a os *slots*
- Aplicar novos estilos dinámicos usando ***v-bind:class*** e ***v-bind:style***

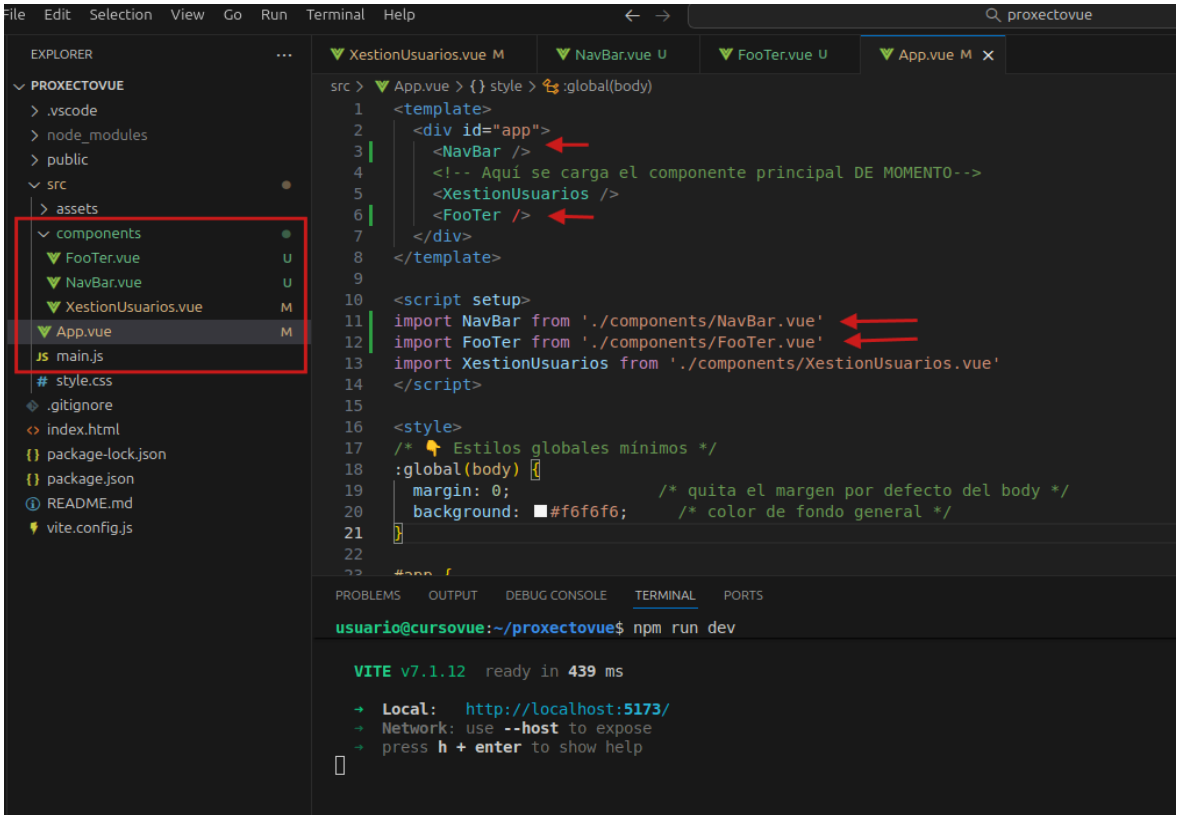
A.1. Compoñentes fixos

Antes de mergullarnos de cheo no enrutamento, é interesante comezar definindo dous compoñentes que van aparecer en todas as páxinas que o encabezamento ou ***Header*** e o Pé ou ***Footer***. Para nós serán os compoñentes *NavBar.vue* e o *FooTer.vue* situado no almacén desta unidade.

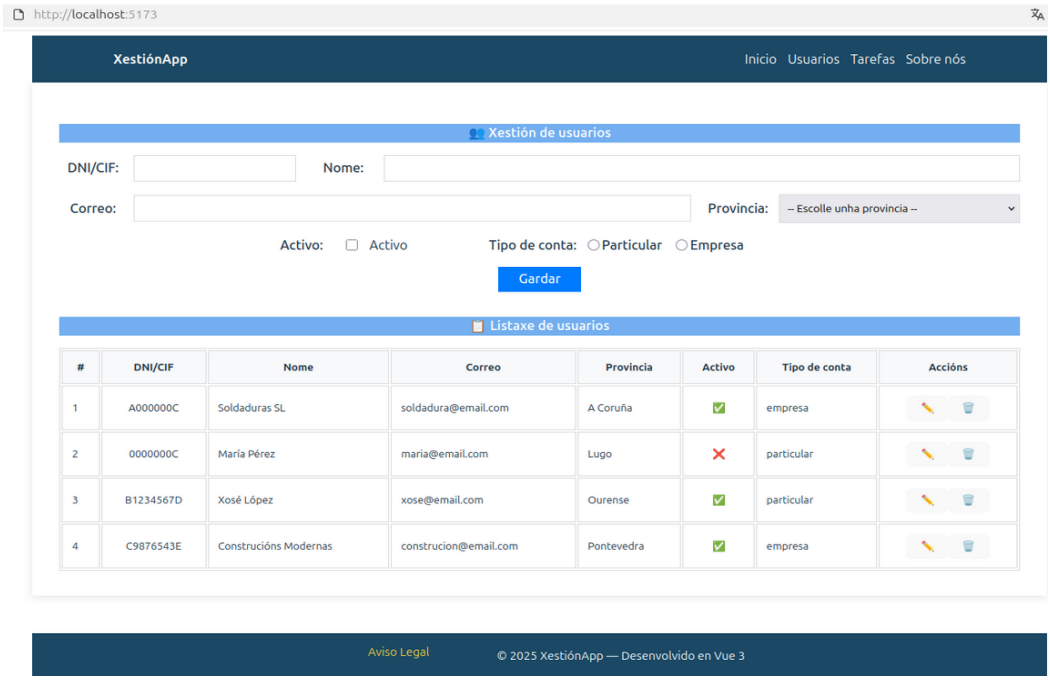


Estes elementos adoitan ser *compoñentes fixos*, xa que non cambian ao navegar entre rutas. Grazas a eles temos **unha estrutura común** en toda a aplicación: un menú na parte superior que permita moverse entre vistas, e un pé de páxina que complete o deseño.

Descargamos estes compoñentes e os engadimos na carpeta ***components*** e facemos as **modificacións** que se sinalan a continuación en ***App.vue.***



O resultado final sería xa se aprecia o *NavBar* ou navegador da aplicación e un típico *FooTer* sinxelo.



O código máis sinxelo é o de *Footer.vue*, onde pode aparecer aspectos como avisos legais, contactos, acerca de....

FooTer.vue M

```
c > components > FooTer.vue > {} template
1   <template>
2     <footer class="footer-content footer">
3       <ul class="link-aviso">
4       <router-link to="/avisolegal" target="_blank" >Aviso Legal</router-link>
5       </ul>
6       <p>© {{ new Date().getFullYear() }} XestiónApp — Desenvolvido en Vue 3</p>
7     </footer>
8   </template>
9
10  <script setup></script>
11
12  <style scoped>
13  .footer {
14    background: #1b4965;
15    color: white;
16    text-align: center;
17    padding: 0.2rem 6rem;
18    margin-top: 3rem;
19    font-size: 0.9rem;
20  }
21
22  .link-aviso {
23    display: block;
24    margin-top: 6px;
25    color: #f9d342;
26    cursor: pointer;
27  }
28
29  .footer-content {
30    display: flex;
31    justify-content: center; /* centra horizontalmente */
32    align-items: center;      /* alinea verticalmente na mesma liña */
33    gap: 5rem;                /* espazo entre o texto e o enlace */
34  }
35
36  </style>
```

O código de NavBar.vue é o seguinte e onde basicamente contén o menú, logo, búsquea e si fora o caso o acceso de usuarios sería este.

NavBar.vue M

src > components > NavBar.vue > { } style scoped > .logo

```vue
<template>
  <nav class="navbar">
    <div class="logo">XestiónApp</div>
    <ul class="menu">
      <li><RouterLink to="/">Inicio</RouterLink></li>
      <li><RouterLink to="/usuarios">Usuarios</RouterLink></li>
      <li><RouterLink to="/tarefas">Tarefas</RouterLink></li>
      <li><RouterLink to="/sobrenos">Sobre nós</RouterLink></li>
    </ul>
  </nav>
</template>

<script setup></script>

<style scoped>
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.1rem 6rem;
  background: #1b4965;
  color: white;
}
.logo {
  font-weight: bold;
  font-size: 1rem;
}
.menu {
  display: flex;
  gap: 0.8rem  ;
  list-style: none;
}
.menu a {
  color: white;
  text-decoration: none;
  font-weight: 500;
}
.menu a.router-link-active {
  border-bottom: 0.2rem solid #ffe66d;
}
</style>
```

En ambos casos aparece a directiva ***router-link*** que é a que nos permite navegar dentro da páxina vendo diferentes vistas da mesma. Para iso necesitamos un elemento dentro de *Vue.js* chamado, *Vue-Router* que permite facer ditas operacións.

A.2. Configuración vue-router

Vue Router é a librería oficial de ***enrutamento*** para aplicacións creadas con Vue.js. Permite converter unha aplicación común nunha **SPA (*Single Page Application*)**, onde a navegación entre "páxinas" non require recargar o navegador.

Con Vue Router podemos:

- Crear **varias vistas** dentro dunha **mesma aplicación.**
- **Definir rutas** como nunha *web* tradicional: /inicio, /clientes, /acercadenos...).
- Navegar entre elas **sen perda de estado**.
- **Traballar con parámetros**, rutas anidadas, protección de rutas, etc.

Nunha SPA real, ao cambiar de páxina ***non se carga outra vez o HTML*****,** só se troca o compoñente que se está mostrando. Vue Router **encárgase de xestionar este proceso.**

Precisa para elo dos seguintes **elementos básicos:**

- ***RouterView*****:** é o compoñente que **di a Vue onde debe *renderizar* a vista actual**.
- ***RouterLink*****:** é o compoñente equivalente aos enlaces ***<a>*** nunha web tradicional, **pero non é unha etiqueta HTML**, senón un compoñente de Vue que evita a recarga da páxina.
- **Historial (*createWebHistory*):** Vue Router xestiona a navegación usando un modo de historial:
  - ***createWeHistory()***: usa a API moderna do navegador *(pushState)*, sen "#".
  - ***createWebHashHistory(***): usa "#", máis compatible pero menos elegante.
- **Parámetros**: permiten rutas dinámicas (/usuario/123...)
- **Navegación programática**: as veces necesitamos abrir ligazóns que non están no menú principal ou ben lanzalo dende un método de JS.

```
import { useRouter } from 'vue-router'

const router = useRouter()

router.push('/inicio')
```

O seguinte esquema conceptual preséntanos de forma más resumida o anterior.

Concepto	Que é	Para que serve
Ruta	Obxecto configuración	Indica que compoñente se mostra nunha URL
RouterLink	Compoñente de navegación	Cambia de páxina sen recargar
RouterView	Zona dinámica	Visualiza o compoñente asociado á ruta actual
Parámetros	Datos na URL	Permiten rutas dinámicas (ex: /usuario/12)
useRoute	Hook	Ler información da ruta (params, query...)
useRouter	Hook	Navegar por código (push, replace...)
Historial	Modo de navegación	Controla como se ve a URL

A utilización de Vue Router, ademais de darlle un aspecto máis profesional, permite unha organización máis limpa do proxecto, traballar con *CRUDs* complexos, autenticación e outras prácticas dende o inicio.

Empecemos pola parte práctica.

Paso 1. Instalación de Vue-Router

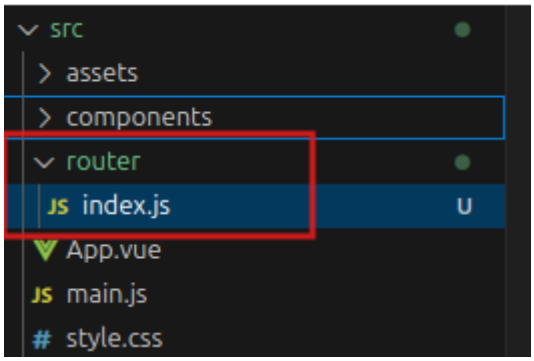
```
usuario@cursovue:~/proxectovue$ npm install vue-router

added 2 packages, and audited 38 packages in 4s

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Paso 2. Crear a carpeta ***router*** e dentro dela o arquivo ***index.js***. Pódese chamar de outra maneira pero ese nome é o habitual.



Paso 3. Dentro de ese arquivo imos a establecer o contido básico seguinte de acordo o noso menú.

index.js U

```
router > JS index.js > default
import { createRouter, createWebHistory } from 'vue-router'
import IniCio from '../components/IniCio.vue'
import XestionUsuarios from '../components/XestionUsuarios.vue'
import XestionTarefas from '../components/XestionTarefas.vue'
import SobreNos from '../components/SobreNos.vue'
import AvisoLegal from '../components/AvisoLegal.vue'

const routes = [
  { path: '/', name: 'inicio', component: IniCio },
  { path: '/usuarios', name: 'usuarios', component: XestionUsuarios },
  { path: '/tarefas', name: 'tarefas', component: XestionTarefas },
  { path: '/sobrenos', name: 'sobrenos', component: SobreNos },
  { path: '/avisolegal', name: 'avisolegal', component: AvisoLegal}
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router
```

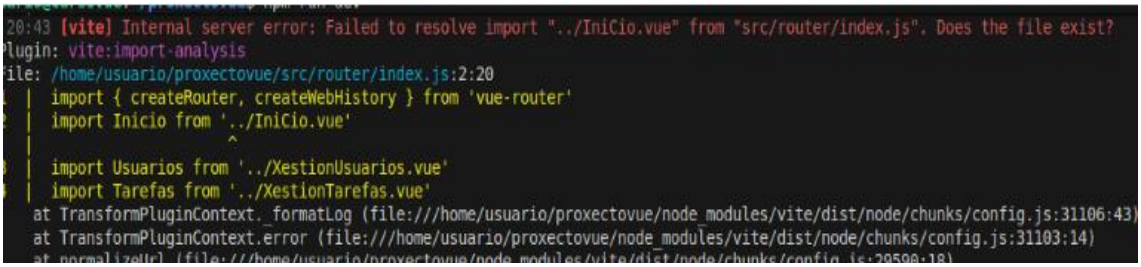
Paso 4. Imos a ***main.js*** para dicirlle a aplicación que ten que usar *router*.

JS main.js

```
import { createApp } from 'vue'
import './style.css'
import App from './App.vue'
import router from './router'

createApp(App).use(router).mount('#app')
```

Antes de seguir avanzando, nestes momentos vemos que o entorno de programación comeza a darnos un erro, en realidade varios, tantos como compoñentes que declaramos e que aínda non codificamos xa que só temos *XestionUsuarios.vue.*



Por iso, imos crear uns compoñentes básicos, de momento sen ningunha funcionalidade e así corrixir os erros e, ademais, permitiranos comprender como funciona *vue-router*. Neste punto **non nos preocupa o contido das páxinas**, senón comprender **como funciona a navegación.**

É importante que os nomes coincidan en maiúsculas e minúsculas, especialmente en *Linux*.

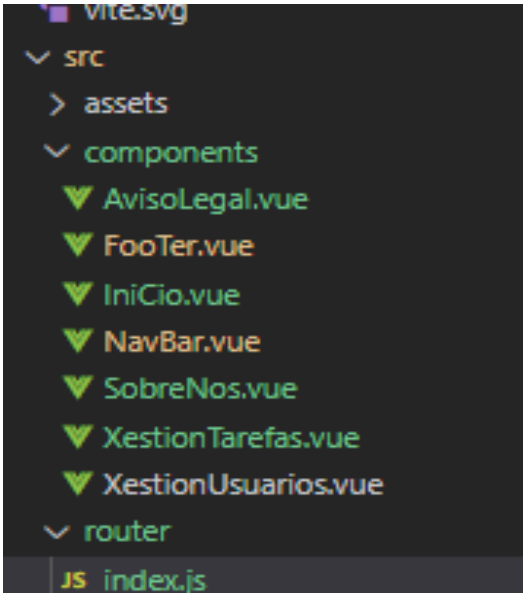
Os código son:

IniCio.vue	XestorTarefas.vue	SobreNos.vue	AvisoLegal.vue
<template> Inicio </template> <script setup></script> <style scope></style>	*<template> Tarefas </template> <script setup></script> <style scope></style>*	*<template> SobreNos </template> <script setup></script> <style scope></style>*	*<template> Aviso Legal </template> <script setup></script> <style scope></style>*

Agora paramos o servidor (Ctrl+C) e volvemos a arrincalo e deixa de dar erros.

En realidade, tampouco é estritamente necesario paralo xa que a medida que facemos modificacións no entorno está continuamente probando se todo funciona. Sen embargo, as veces convén facelo.

A imaxe mostra como debería ser a disposición final.



```
usuario@cursovue:~/proxectovue$ npm run dev

  VITE v7.1.12  ready in 408 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Paso 5. Quedamos modificar o **App.vue**. Ata agora, chamaba directamente a ***XestionUsuarios.vue,*** pero non é que queremos, o que buscamos e que cargue a páxina inicial "/" e que poña "a funcionar" o ***enrutamento***, cargando a función ***router-view.*** **Para mellorar o deseño** fai os cambios que se indican na zona ***style*** o ben podes tomar o arquivo ***App.vue*** do almacén da Unidade.

App.vue M

App.vue > { } style

```vue
<template>
  <div id="app">
    <NavBar />
     <!-- Cargamos o xestor de rutas-->
    <main  >
      <router-view />
    </main>
    <FooTer />
  </div>
</template>

<script setup>
import NavBar from './components/NavBar.vue'
import FooTer from './components/FooTer.vue'
</script>

<style>
body {
  margin: 0;
  background: #f6f6f6;
}

#app { width: 80%;  margin: 0 auto;}

.navbar,
.footer {
  position: fixed;
  left: 50%;
  transform: translateX(-50%);
  width: 80%;
  z-index: 1000;
}

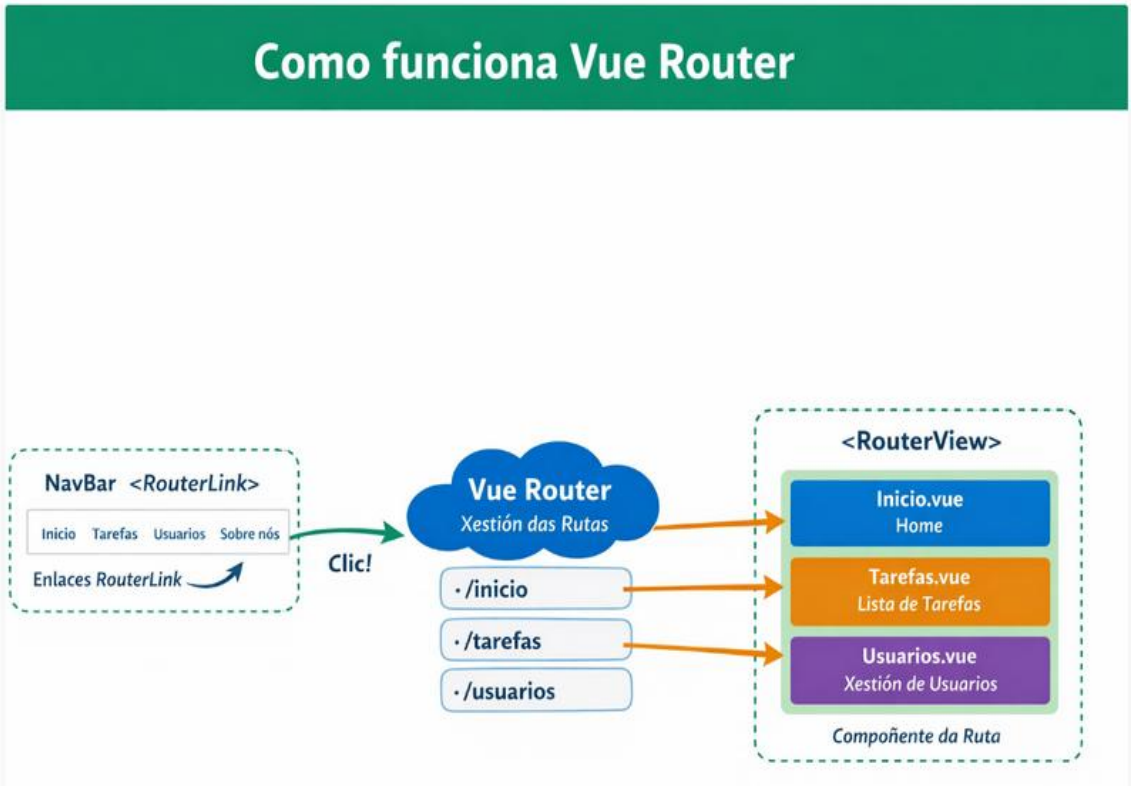
.navbar {top: 0; }

.footer {bottom: 0; }

main { margin-top: 30px; margin-bottom: 30px; }
</style>
```

Unha vez feito, xa podemos navegar polos diferentes ligazóns da aplicación.

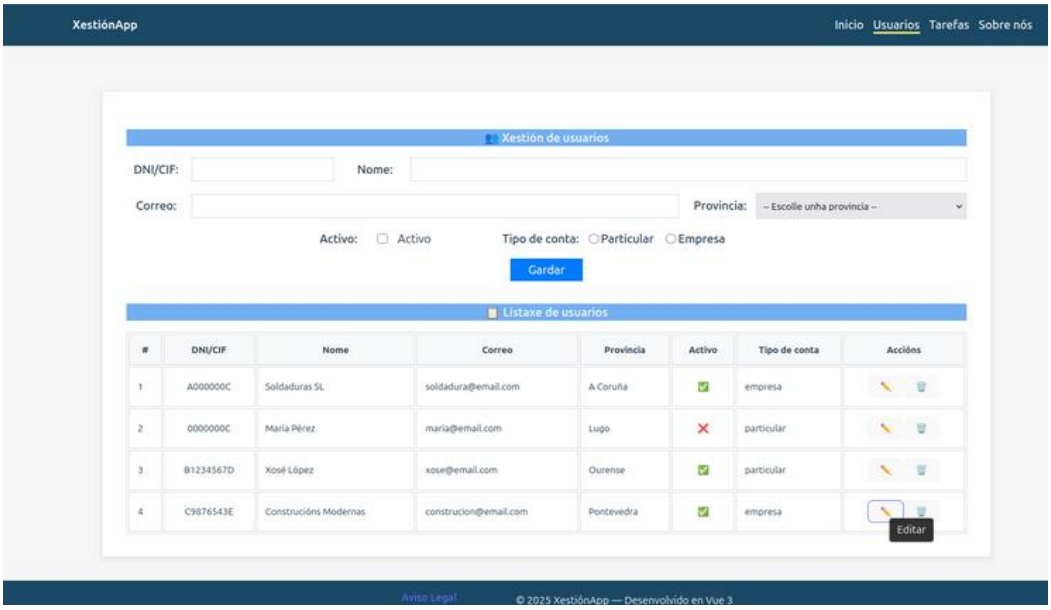
Nas seguinte imaxes móstranos como funciona Vue Router.

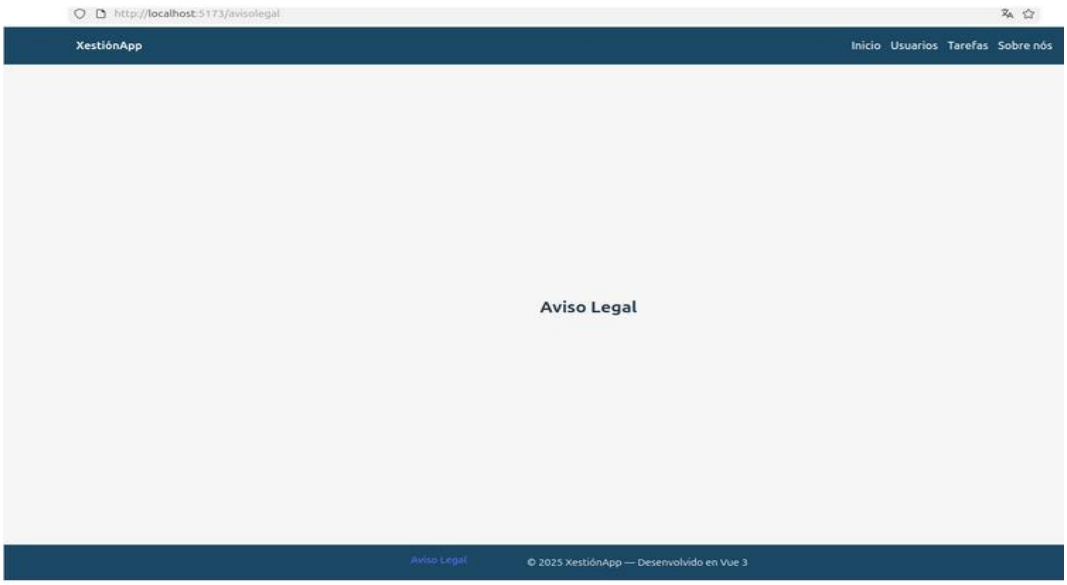


Basicamente, cada vez que pulsamos un ligazón *(router-link),* a librería *Vue Router* busca en ***index.js*** a que compoñente da ruta se corresponde para que *Vue* poda chamalo e cargalo.

As páxinas vanse cargando segundo as vamos chamando.







En resumo, podemos pensar en ***<router-view***> como unha “ventá” que cambia de contido segundo a ruta activa.

A.3. Comunicación entre compoñentes

A **comunicación entre compoñentes en Vue** refírese ao intercambio de datos e eventos entre distintos compoñentes dunha aplicación, permitindo que un compoñente "*pai*" pase información aos seus "*fillos*" mediante ***props***, e que estes envíen de volta eventos usando ***$emit.***

- **Compoñente Pai:** É o que ten o control dos datos principais. Exemplo: *XestionUsuarios.vue* que contén a lista de usuarios.
- **Compoñente Fillo:** Recibe datos do pai e pode facer operacións sobre eses datos ou amosalos. Exemplo: *XestionTarefas.vue* que amosa as tarefas dun usuario concreto.

O *fillo* **non modifica directamente** os datos do *pai*, só recibe información ou envía eventos para que o pai actúe.

As **ferramentas** que usamos en Vue **para a comunicación** son:

Concepto – Directiva	Función
props	Permite ao pai pasar datos ao fillo. *Exemplo: <Fillo :usuario="usuarioSeleccionado"/>*
$emit	Permite ao fillo enviar eventos ao pai. *Exemplo*: *emit('tarefaActualizada', novaTarefa)*
v-model	Pode usarse en compoñentes fillos para crear unha comunicación bidireccional simplificada (vincula prop + evento)
computed	Útil no pai para calcular datos derivados. *Exemplo, contar tarefas dun usuario*
v-for	No fillo, para *renderizar* listas de datos que recibe do pai
v-if	Mostrar elementos no fillo só se hai datos dispoñibles

Imos a entrar en materia.

No **almacén da Unidade 3** temos o compoñente *XestiónTarefas.vue* co código que nos permite mostrar as tarefas dun usuario. De momento, son todas iguais. Cando empecemos na UD4 entón poderemos acercarnos a un suposto máis real.

Empezamos **modificando o compoñente pai ou XestiónUsuarios.vue.** A idea é que cando premamos e carguemos os datos un usuario no formulario, o compoñente fillo *XestionTarefas.vue* cargue as súas tarefas.

- **Zona script**. Engadimos dúas cousas
  - unha referencia para o usuario seleccionado e as súas tarefas no pai
  - aproveitando o botón do lapis ou editar cargar os datos de usuario en *usuario seleccionado* e as *tarefas*

```
///  referencias ao usuario seleccionado e as súas tarefas

const usuarioSeleccionado = ref(null)
const tarefasUsuario = ref([])


/// Zona de métodos ou funcións

function gardarUsuario() {
  usuarios.value.push({ ...novoUsuario })  //engade o novo usuario á lista (copia do obxecto)
  Object.assign(novoUsuario, { nome: "", correo: "", idade: null, provincia: "", activo: false, se
}

function eliminarUsuario(index) {
  usuarios.value.splice(index, 1);   //elimina o usuario da lista
}

function editarUsuario(index) {
  const usuario = usuarios.value[index];   //carga os datos do usuario elixido no formulario
  Object.assign(novoUsuario, usuario);  // carga os datos do usuario no formulario recorda v-model
   // Actualizamos a comunicación co fillo

  usuarioSeleccionado.value = usuario
  tarefasUsuario.value = [
    // Aquí poñemos tarefas de exemplo ou as que veñan da base de datos
    { titulo: "Tarefa 1", dataLimite: "2025-12-31", completada: false },
    { titulo: "Tarefa 2", dataLimite: "2026-01-15", completada: true }
  ]

}

</script>
```

- **Zona Template:** engadimos a referencia a *XestionTarefas.vue.* Isto obríganos a que na zona de script, teñamos que importar o compoñente fillo.

```
  </div>
  <!-- Aquí engadimos o fillo -->
<XestionTarefas
  :usuario="usuarioSeleccionado"
  :tarefas="tarefasUsuario"
/>
</template>

<script setup>
/// Zona de declaracións
import { ref, reactive, onMounted } from 'vue'
// Importamos o compoñente fillo
import XestionTarefas from './XestionTarefas.vue'

const usuarios = ref([])  //almacena la lista de usuarios e os seus cambios

const novoUsuario = reactive({
  dni: "",
  nome: "",
  correo: "",
  provincia: "",
  activo: false,
  tipoCuenta: ""
})
```

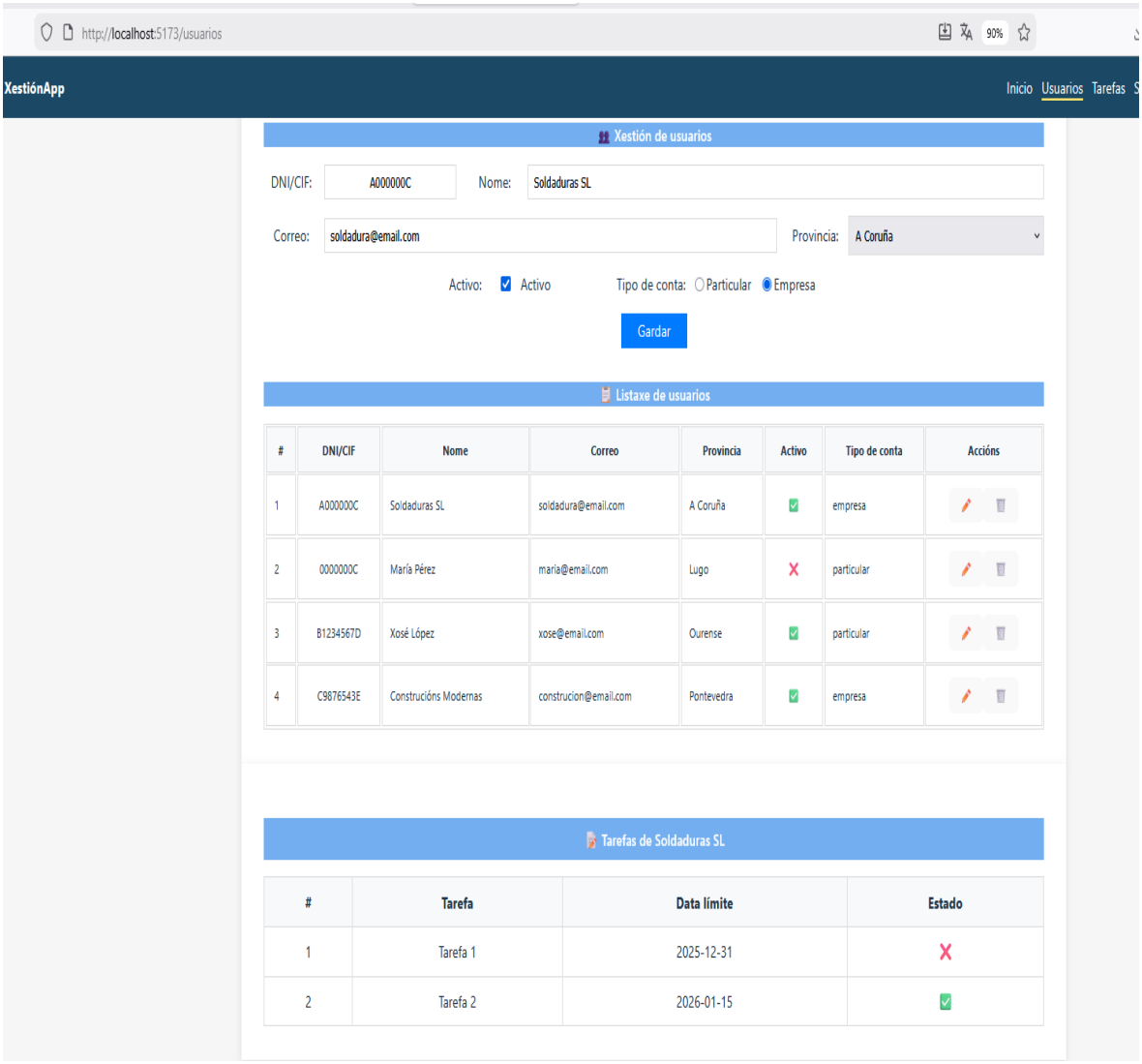
Estas últimas liñas é o que conecta o **compoñente pai** *(XestionUsuarios.vue*) co **fillo** (*XestionTarefas.vue*) mediante **props**, que son como "canles de comunicación de datos unidireccionais".

- ***:usuario="usuarioSeleccionado":*** está a pasar un obxecto usuarioSeleccionado do pai ao fillo. O ":" é un ***shorthand* (expresión curta)** de **v-*bind***, que indica que o valor é unha expresión de *JavaScript* (non un *string* literal). No fillo, podemos acceder a ese obxecto directamente co nome ***usuario***.
- ***:tarefas="tarefasDoUsuario":*** de igual forma, pasa ao fillo a lista de tarefas correspondente ao usuario seleccionado. No fillo, está dispoñible como tarefas.

Porque é importante:

1. Permite que o fillo **se manteña reactivo**: cada vez que o pai cambia *usuarioSeleccionado* ou *tarefasDoUsuario,* o fillo actualízase automaticamente sen que o fillo teña que facer nada.
2. Mantén unha separación de responsabilidades: o pai controla os datos e o fillo só os amosa.
3. Facilita o fluxo de comunicación ***unidireccional***, que é a forma recomendada en Vue para evitar problemas de sincronización ou efectos colaterais.

Aínda que non é unha limitación técnica, senón unha consecuencia do fluxo de datos entre compoñentes pai → fillo, o uso de ***props*** implica que **temos que renderizar no mesmo compoñente, neste caso o pai, o compoñente fillo, neste caso as tarefas** o que as veces non é algo visualmente atractivo, como se pode ver na imaxe.



Ata agora vimos comunicación **de pai a fillo**. Agora imos ver que pasa cando o fillo necesita **avisar ao pai** de que ocorreu algo usando ***$emit***.

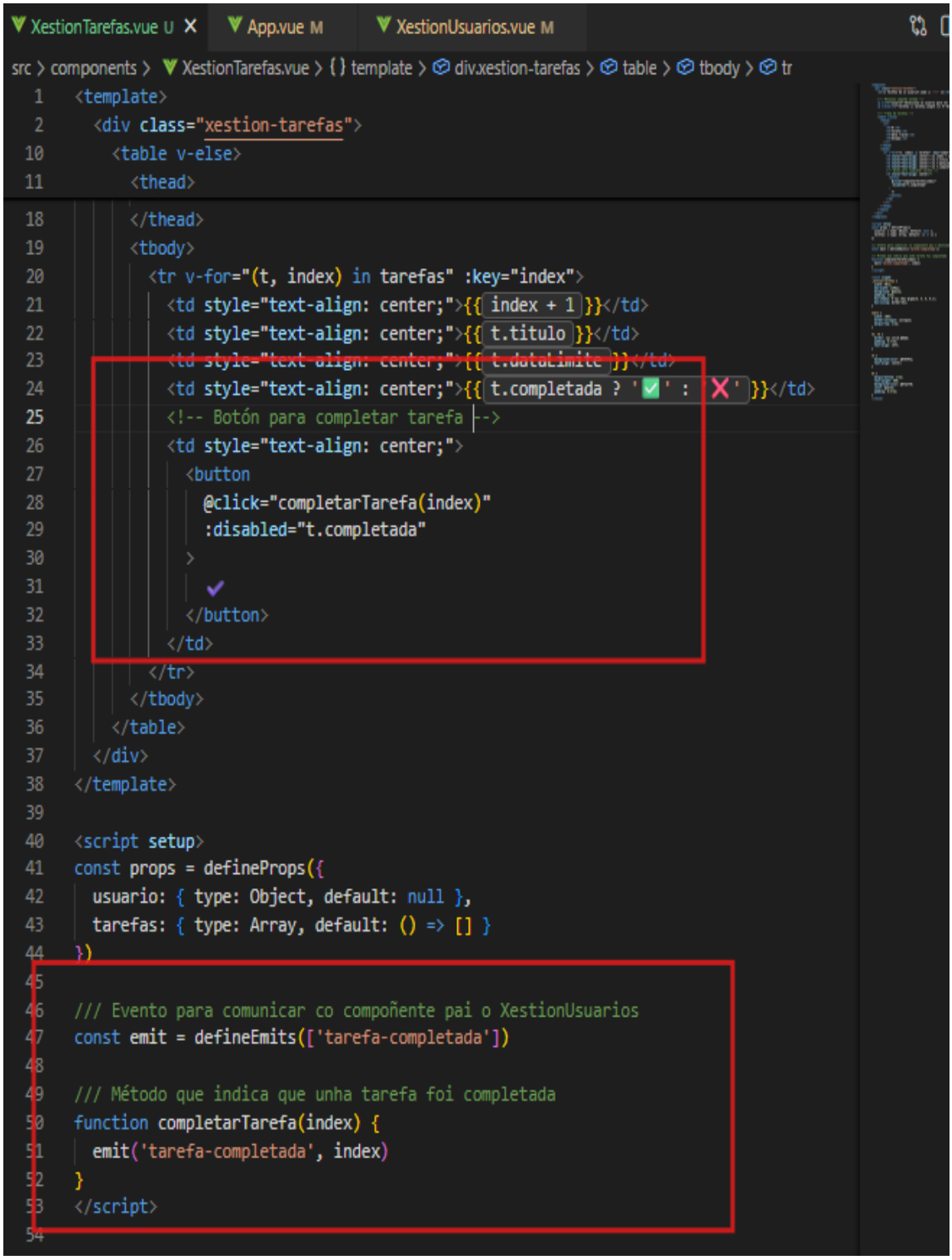
O compoñente *XestiónUsuarios.vue* será o pai, pero tamén recibirá sinais do fillo medíante a sinal ***$emit.***

Faremos algo sinxelo e realista:

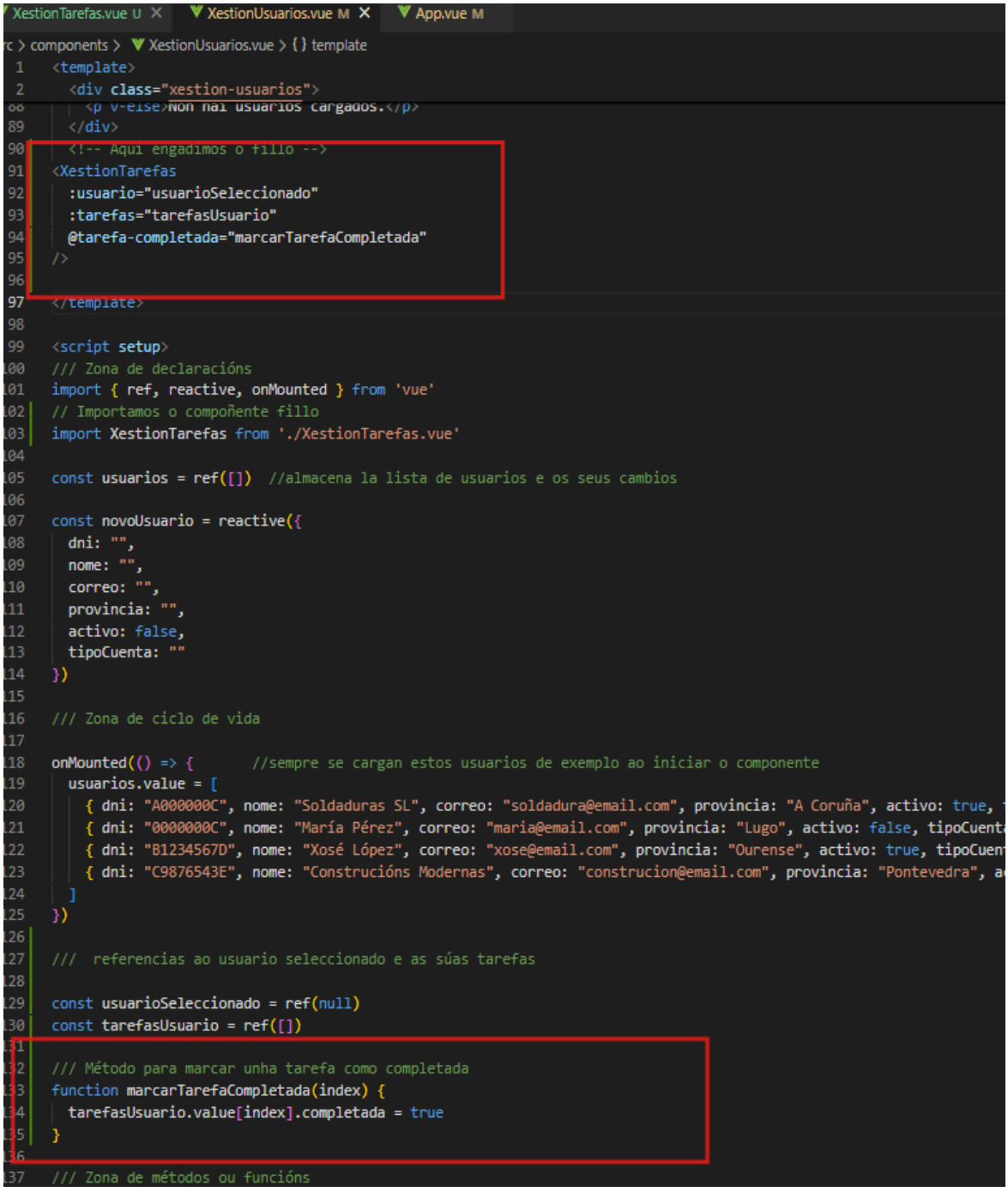
- **Marcar unha tarefa** como completada desde *XestionTarefas.vue*
- O fillo emite: *"esta tarefa cambiou"*

- O pai decide que facer con ese cambio

Para iso en *XestiónTarefas.vue* só engadimos o seguinte que permite que o compoñente fillo envíe unha sinal ao compoñente pai o *XestiónUsuarios.vu.*

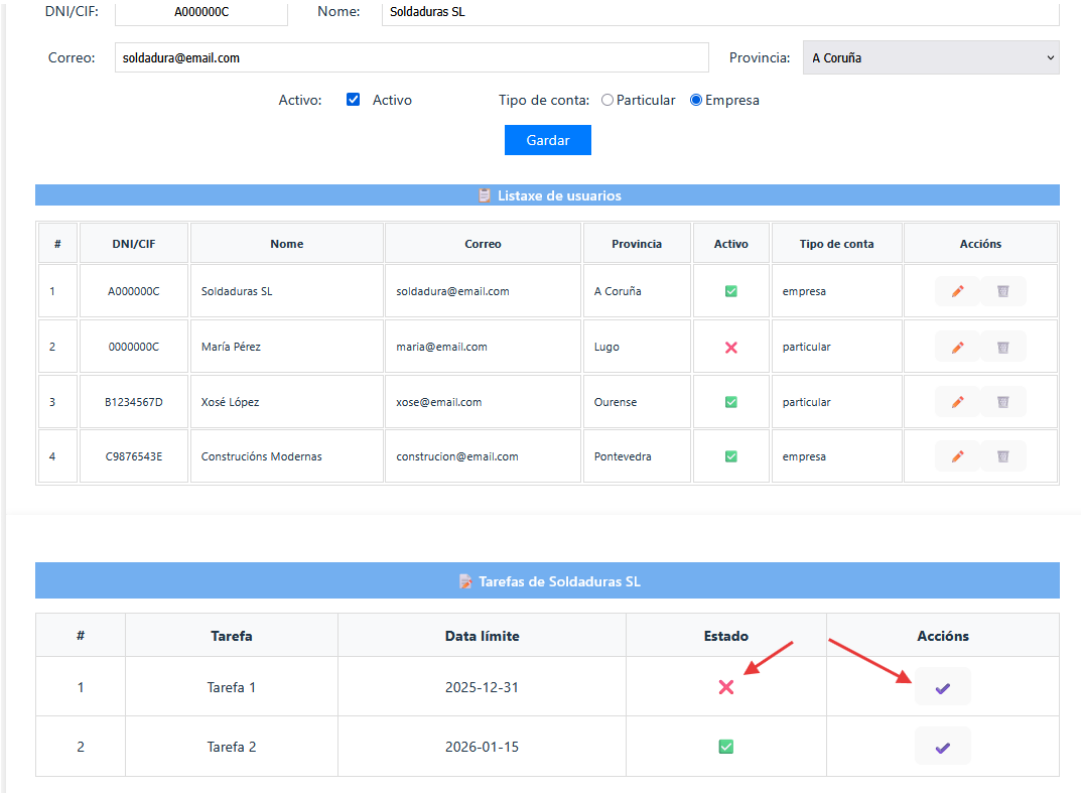


Mentres, no compoñente *XestionUsuarios.vue* engadimos a función marcar a tarefa completada.

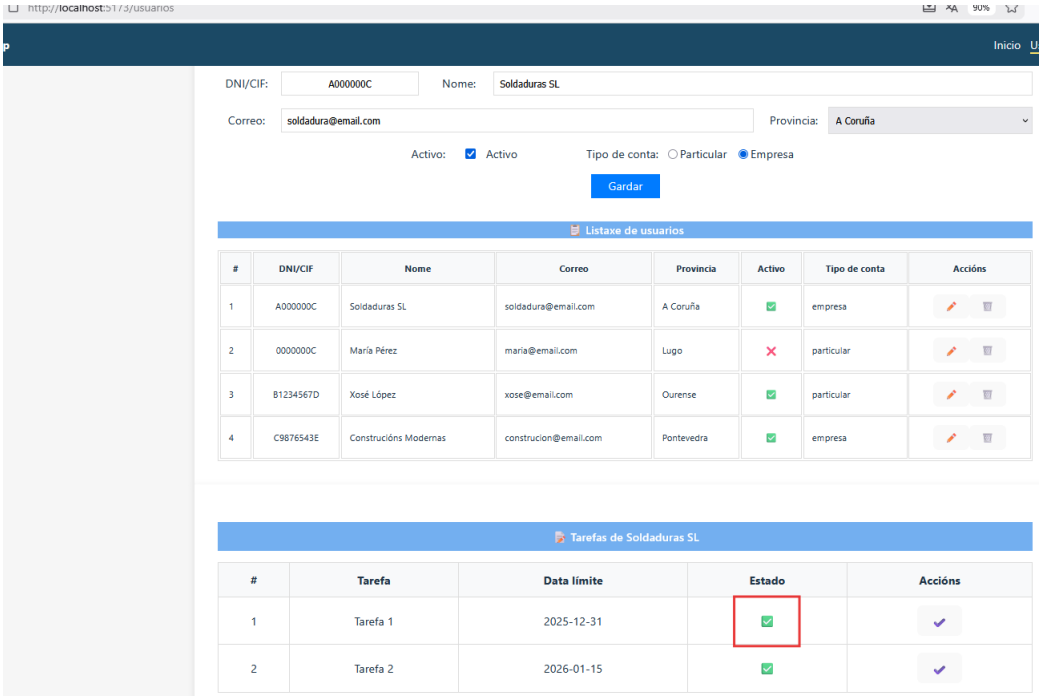


XestionUsuarios.vue é o **pai**, porque ten os datos reais. *XestionTarefas.vue* é o **fillo**, porque só amosa e interactúa. O fillo **non cambia datos directamente**. Cando pasa algo importante usa ***$emit,*** e o pai decide como actualizar o estado da tarefa.

Nas imaxes posteriores vemos o funcionamento.



Se pulsamos en Accións na Tarefa 1, un evento dende o compoñente fillo rexistrase no compoñente pai.



A.4. Resumo

- **Enrutamento en *Vue***: Comprender e aplicar rutas para navegar entre compoñentes dentro dunha *SPA*.
- **Rutas comodín**: Xestionar *URLs* inexistentes mostrando un compoñente de "Páxina non encontrada".
- **Comunicación pai → fillo**: Pasar datos mediante ***props*** para amosar información adicional nos compoñentes fillos.
- **Estado básico compartido**: Mantemento de datos seleccionados e actualización de compoñentes relacionados sen ***backend***.